

MeasurementDisplay.svelte: Real-time respiratory rate display component

Project reference: STN22_Thoracic belt for respiratory rate measurement

Authors: Hakkou Dounia

Date: 2025–2026 © SIS TEAM NANCY

File header

```
<!--
Description: Svelte component for real-time respiratory rate display.
Receives a Patient prop from the parent component.
Manages session lifecycle (start / stop) via the sessionsAPI.
Polls the backend every second to retrieve the latest RPM measurement
and displays it live. Cleans up the polling interval on destroy.

Project reference : STN22_Thoracic belt FR
Authors: Hakkou Dounia
Date: 2025-2026
© SIS TEAM NANCY
-->
```

Script block (TypeScript)

```
<script lang="ts">
  // Import session API helpers (start / stop) and the Patient type definition
  import { sessionsAPI, type Patient } from './api';
  // onDestroy: Svelte lifecycle hook called when the component is removed from the DOM
  // Used here to clear the polling interval and prevent memory leaks
  import { onDestroy } from 'svelte';

  // Null when no patient has been selected yet.
  export let patient: Patient | null = null;

  // Current respiratory rate in RPM. Null while waiting for the first measurement.
  let currentRPM: number | null = null;
  // ID of the active session returned by the backend after startSession().
  let sessionId: string | null = null;
  // Boolean flag: true while a recording session is active.
  let isRecording = false;
  // Reference to the setInterval timer used for polling. Stored to allow cancellation.
  let pollingInterval: ReturnType<typeof setInterval> | null = null;

  // startSession
  // Called when the user clicks 'Demarrer'.
  // Guards against missing patient, calls the API, then starts polling.
  async function startSession() {
    // Safety guard: a patient must be selected before starting a session
    if (!patient) {
      alert("Selectionnez un patient d'abord !");
      return;
    }

    try {
      // POST /set-patient: registers the active patient on the backend
      // Returns a sessionId that links all incoming measurements to this session
      const data = await sessionsAPI.start(patient.id);
      sessionId = data.sessionId;
      isRecording = true;    // Switch UI to recording state
      currentRPM = null;    // Reset display while waiting for first measurement

      // Launch the polling loop that fetches RPM every second
      startPollingMeasurements();
      console.log('Session demarree:', sessionId);
    } catch (err: any) {
      // Most common cause: the Node.js backend server is not running
      console.error('Erreur demarrage session:', err);
      alert('Erreur: Le serveur doit etre demarre');
    }
  }
}
```

```

// stopSession
// Called when the user clicks 'Arreter'.
// Stops the backend session, clears the polling interval, resets state.
async function stopSession() {
  if (!sessionId) return; // Nothing to stop if no active session

  try {
    // POST /stop-session: marks the session as ended on the backend
    await sessionsAPI.stop();
    isRecording = false;

    // Cancel the polling interval to stop fetching measurements
    if (pollingInterval) {
      clearInterval(pollingInterval);
      pollingInterval = null;
    }

    sessionId = null;
    console.log('Session terminee');
  } catch (err: any) {
    console.error('Erreur arret session:', err);
  }
}

// startPollingMeasurements
// Sets up a 1-second interval that fetches the latest RPM measurement
// for the active session from the backend REST API.
function startPollingMeasurements() {
  if (!sessionId) return;

  // Use environment variable for the API base URL; fall back to production URL
  const API_URL = import.meta.env.VITE_API_URL || 'https://sisteamnancy.fr/api-respiramed';

  // setInterval fires the async callback every 1000 ms
  pollingInterval = setInterval(async () => {
    try {
      if (!sessionId) return; // Guard: session may have been stopped

      // GET /measurements/:sessionId/latest returns the most recent RPM value
      const response = await fetch(`${API_URL}/measurements/${sessionId}/latest`);

      if (response.ok) {
        const data = await response.json();
        // Parse the rpm field and update the reactive variable to trigger UI re-render
        if (data && data.rpm) {
          currentRPM = parseFloat(data.rpm);
        }
      }
    } catch (err) {
      console.error('Erreur polling measurements:', err);
    }
  }, 1000);
}

// onDestroy
// Svelte calls this callback when the component is unmounted from the DOM.
// Clearing the interval here prevents it from continuing to fire after
// the component is gone, which would cause memory leaks and stale updates.
onDestroy(() => {
  if (pollingInterval) {
    clearInterval(pollingInterval);
  }
});
</script>

```

HTML template (Svelte)

```

<!-- Root container for the measurement UI -->
<div class="measurement-container">

```

```

<!-- Conditional rendering: show UI only if a patient is selected -->
{#if patient}

  <!-- Placeholder div for the future respiratory curve chart -->
  <!-- A Chart.js or D3.js graph should replace this element -->
  <div class="graph-placeholder">
    <p>Courbe a mettre</p>
  </div>

  <!-- RPM display box: shows current value or a waiting message -->
  <div class="rpm-box">
    {#if currentRPM !== null}
      <!-- toFixed(1): format with one decimal place (e.g. 15.3 RPM) -->
      <div class="rpm-value">{currentRPM.toFixed(1)} <span>RPM</span></div>
    {:else}
      <!-- Shown before the first measurement arrives from the backend -->
      <div class="rpm-value waiting">En attente des donnees...</div>
    {/if}
  </div>

  <!-- Control buttons: toggles between Start and Stop based on isRecording -->
  <div class="controls">
    {#if !isRecording}
      <!-- Start button: triggers startSession() on click -->
      <button class="start-btn" on:click={startSession}>Demarrer</button>
    {:else}
      <!-- Stop button: triggers stopSession() on click -->
      <button class="stop-btn" on:click={stopSession}>Arreter</button>
    {/if}
  </div>

{:else}
  <!-- Fallback message when no patient is selected -->
  <p>Selectionnez un patient pour demarrer la mesure.</p>
{/if}
</div>

```

CSS styles (scoped)

```

<style>
  /* Scoped styles — only apply to this component's elements */

  /* Main container: centered column layout with consistent spacing */
  .measurement-container {
    display: flex; flex-direction: column; align-items: center;
    gap: 1.5rem; padding: 2rem;
    max-width: 400px; margin: 2rem auto;
    font-family: 'Inter', sans-serif;
  }

  /* Placeholder for the future respiratory chart */
  .graph-placeholder {
    width: 130%; height: 150px;
    background: #f3f4f6; border-radius: 12px;
    display: flex; align-items: center; justify-content: center;
    color: #9ca3af; font-size: 1rem; font-weight: 500;
  }

  /* RPM display box: gradient background, large bold text */
  .rpm-box {
    width: 100%; border-radius: 12px; padding: 2rem;
    background: linear-gradient(135deg, #7b41e1 0%, #93c5fd 100%);
    display: flex; justify-content: center; align-items: center;
    color: white; font-size: 2.5rem; font-weight: 700;
    box-shadow: 0 8px 20px rgba(0,0,0,0.15);
  }

  /* Start button: green tone */
  .start-btn { background: #79d8b9; color: white; }
  .start-btn:hover { background: #5fc9a8; }

  /* Stop button: muted red tone */
  .stop-btn { background: #ba8383; color: white; }

```

```
.stop-btn:hover { background: #a86f6f; }

/* Responsive layout for small screens (mobile) */
@media (max-width: 480px) {
  .measurement-container { padding: 1rem; max-width: 100%; }
  .graph-placeholder { width: 100%; }
  .rpm-box { font-size: 2rem; }
  .controls { flex-direction: column; width: 100%; }
  button { width: 100%; }
}
</style>
```